

Отчет

Примененные методологии и шаблоны проектирования

Ниже представлены техники, которые применены в макете веб-приложения.

Методология	Преимущества	Источники
Компонентный дизайн Сайт-шаблон Каждый компонент, его код, разбиты на атомарные компоненты, вложенные друг в друга	Компоненты можно переиспользовать в разных частях сайта, если они достаточно гибкие, например, компоненты “Кнопка” или “Список”. Создание новых страниц происходит быстрее, так как не нужно заново проектировать и прописывать одни и те же компоненты снова и снова.	https://www.geeksforgeeks.org/system-design/what-is-component-design/
MSD Модифицированный вариант FSD (Feature-Sliced design), придуманный в Сбере. Позволяет разделить разные части приложения, в том числе бизнес-логику и интерфейс, на независимые слои.	Бизнес-логика, при правильной организации, не зависит от реализации интерфейса, то есть оба могут быть переписаны без влияния друг на друга. Код из слоя shared/ может быть переиспользован в других проектах в исходном виде.	https://habr.com/ru/companies/sberbank/articles/959400/
CSR, Весь рендеринг, а Client-side API происходят на клиенте rendering	Ускоряет разработку за счет большей нагрузки на клиент. Пользователь сразу видит элементы интерфейса за счет их загрузки прямо перед его глазами, например, изображений с сайта picsam. Так он не подумает, что сайт ему не отвечает, что может быть при SSR	https://www.patterns.dev/react/client-side-rendering/

Шаблон	Преимущества	Пример использования в проекте	Источники
Фабрика - место создания объектов с одинаковыми полями снова и снова мы используем специальную функцию - "фабрику объектов" - в которую на вход подаем значения полей и на выходе получаем тот же объект	Код соответствует методологии DRY (don't repeat yourself) Фабрика является единым источником правды, потому нужно изменить только ее реализацию, а не поля каждого отдельного объекта, где можно что-то забыть	При создании псевдо-пользователей используется функция <code>userFabric()</code>	https://www.patterns.dev/vanilla/factory-pattern/
Провайдер - задача пропсов от родительских компонентов до дочерних	Родительские компоненты могут влиять на состояние дочерних, передавая им параметры Поля форм могут быть контролируемыми (тоже шаблон в React)	Везде	https://www.patterns.dev/vanilla/provider-pattern/

Шаблон	Преимущества	Пример использования в проекте	Источники
Глобальное состояние переменных, через которые нужно использовать в op-text местах в приложении, выносятся в "глобальное состояние", к которому можно обратиться из любого компонента без необходимости передачи пропсов от родителей к детям	Позволяет избежать каскадной передачи глобального состояния от родителя ко всем детям, что уменьшает количество пропсов на вход и делает код менее сложным. Промежуточные компоненты не получают ненужные им данные, отчего код становится чище и более безопасным, ведь они (компоненты) не могут менять пропсы, пока состояние не будет импортировано явно	Пользователь (@me-myao), от лица которого продемонстрирована работа прототипа, хранится в CurrentUserContext	https://www.patterns.dev/dev/vanilla/provider-pattern/
Статическая импортировка (require(), которая уменьшает размер бандла проекта и за счет этого делает приложение быстрее)	Размер бандла уменьшается, то есть на клиент нужно передавать файлы меньшего размера. Приложение работает быстрее	Везде	https://www.patterns.dev/dev/vanilla/static-import/

Шаблон	Преимущества	Пример использования в проекте	Источники
Hook-Хуки (с англ. «рючки») позволяют обращаться к конкретным фичам React, а также самого приложения, через специальные функции	Логика конкретных фич, например, использования куки, или взаимодействия с конкретным API, логически вынесена в одну функцию, которую можно применить в любом компоненте без передачи в пропсах и без значимого увеличения размера бандла	Везде, где используются хуки React. В дальнейшем будут разработаны собственные хуки для разделения ответственности между компонентами и конкретными фичами	https://www.patterns.dev/react/hooks-pattern/

Разработка интерактивного макета

Ниже рассматриваются далеко не все страницы макета, а только ключевые для бизнеса.

Лента мемов

- **Компонентный дизайн:** страница разделена на боковую панель и на ленту мемов. Лента состоит из компонента MemePost, которые можно пролистывать, причем, этот компонент используется и как самостоятельная динамическая страница для маршрута `/post/:postId`, то есть для просмотра конкретного поста в отрыве от ленты `/feed`. Этот же компонент состоит из компонента с данными о посте (левый-нижний угол), изображения поста по центру и кнопок действий справа.

Профиль пользователя

- **Компонентный дизайн:** страница разделена на боковую панель, заголовок профиля со статистикой и кнопками действия (ниже - только настройки профиля, так как это профиль *данного* клиента), а также основное содержимое, которое меняется при переключении между вкладками.

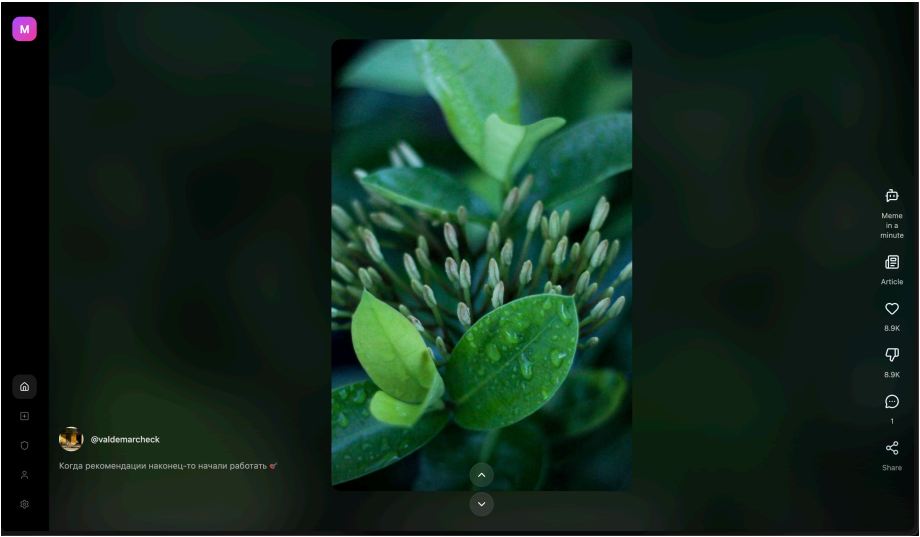


Рис. 1: 600

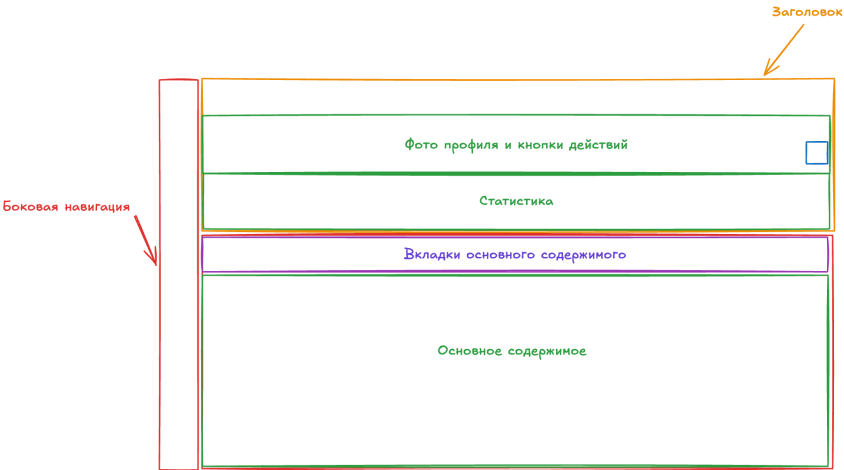
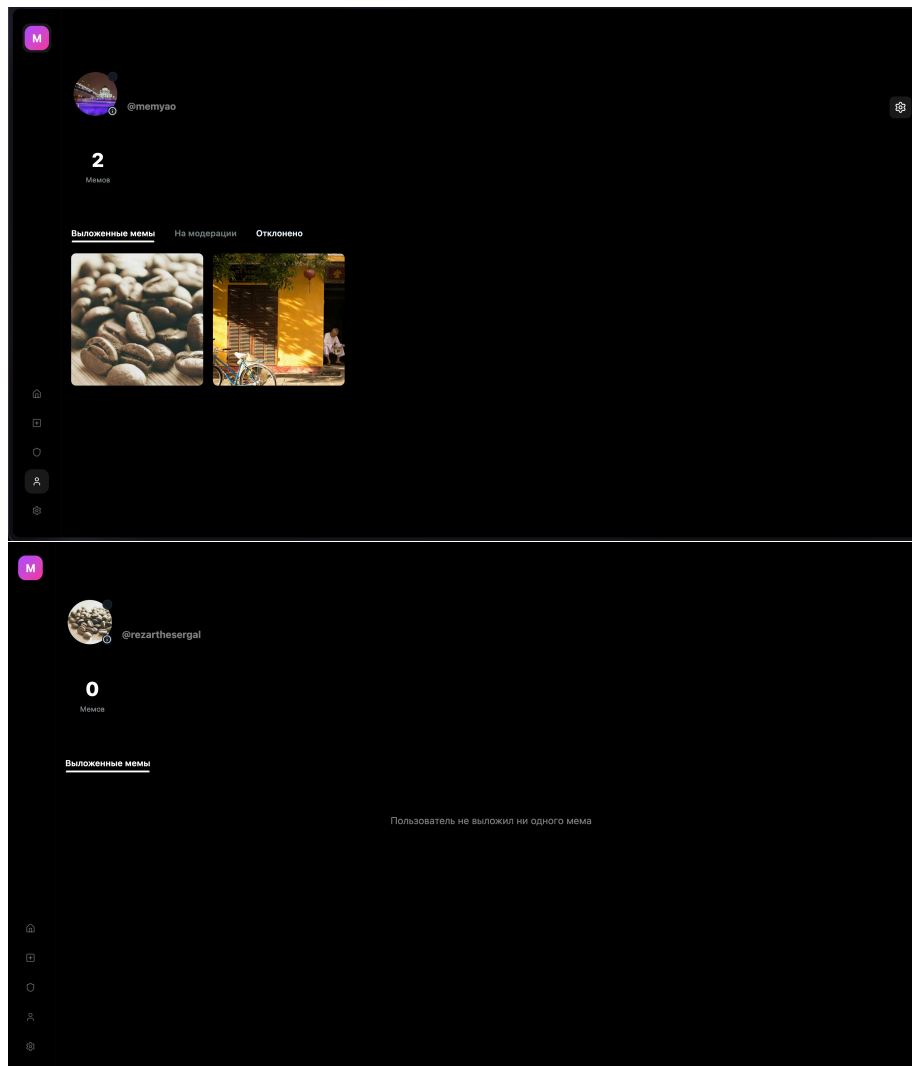
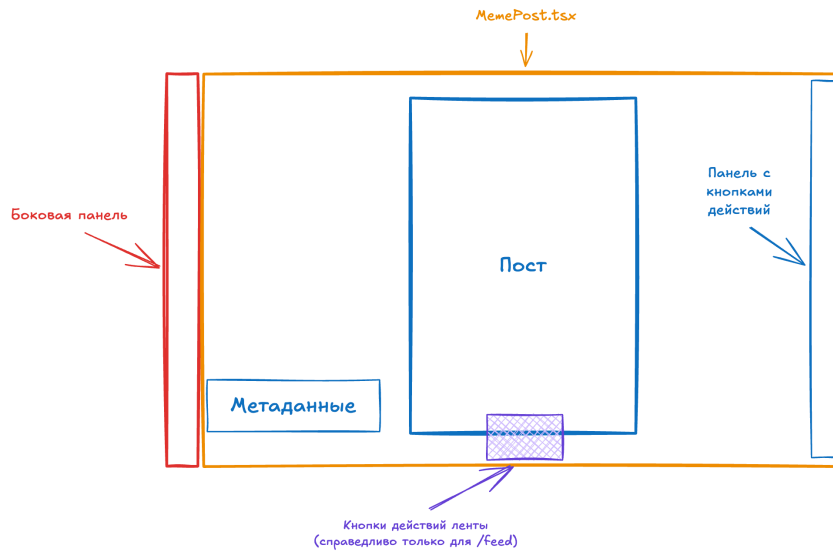


Рис. 2: 600

- **Глобальное состояние:** компонент страницы выглядит по-разному в зависимости от того, от лица какого пользователя мы просматриваем на ту или иную страницу. Если это *наша* страница, то мы можем зайти в настройки профиля на панели действий (рис. 1), а также мы можем просмотреть, какие посты находятся на модерации и какие были отклонены. Если это *чужая* страница, такой кнопки нет (рис. 2).





Сравнение технологий

Мы рассматривали React, Vue и Angular.

Технология	Преимущества	Недостатки
React	Четкое разделение JS, CSS и HTML в структуре проекта Огромное количество обучающих материалов и библиотек	Крутая кривая обучения
Vue	JS, CSS, HTML конкретного компонента находятся в одном файле Интуитивная система хуков и состояний	Сравнительно небольшое количество обучающих материалов и библиотек
Angular	Полноценный фреймворк со встроенными функциями (роутинг, HTTP-клиент, формы) Мощная экосистема Angular CLI	Очень крутая кривая обучения из-за сложных концепций (dependency injection, модули, декораторы) Большой размер бандла и меньшая производительность по сравнению с React

React является оптимальным выбором для разработки нашей платформы

по следующим причинам:

1. **Гибкость и модульность** — React это библиотека, а не полноценный фреймворк, что дает свободу в выборе архитектуры и дополнительных инструментов;
2. **Производительность** — React использует виртуальный DOM и имеет наименьший Speed Index (0.8s) по сравнению с Angular (1.2s) и Vue (1.7s), что важно для быстрой загрузки мемов;
3. **Компонентный подход** — идеально соответствует применяемой методологии компонентного дизайна. Компоненты MemePost, боковая панель и другие элементы легко переиспользуются;
4. **Огромная экосистема** — для React существует наибольшее количество библиотек и обучающих материалов, что ускоряет разработку и решение возникающих проблем;
5. **Масштабируемость** — React легче масштабировать и интегрировать с другими технологиями;
6. **Меньший размер бандла** — по сравнению с Angular, React имеет меньший размер бандла, что обеспечивает более быструю загрузку приложения для пользователей;
7. **Поддержка применяемых паттернов** — React нативно поддерживает все используемые в проекте паттерны: хуки, контекст (CurrentUser-Context), провайдеры;

Angular был бы избыточен для данного проекта — это полнофункциональный фреймворк, который требует изучения множества концепций (dependency injection, модули, декораторы) и имеет больший размер бандла.

Vue хоть и интуитивен, но имеет меньшее сообщество и меньше готовых решений, что может замедлить разработку в долгосрочной перспективе.